

Internship : Data Analyst Internship

Task 6 - Sales Trend Analysis Using Aggregations

Objective : Analyze monthly revenue and order volume

Database : MySQL

Name : Himani Khawashi

Step 1: Create Database

```
CREATE DATABASE SALES_ANALYSIS;
```

Step 2: Use Database

```
USE SALES_ANALYSIS
```

Step 3: Create Table

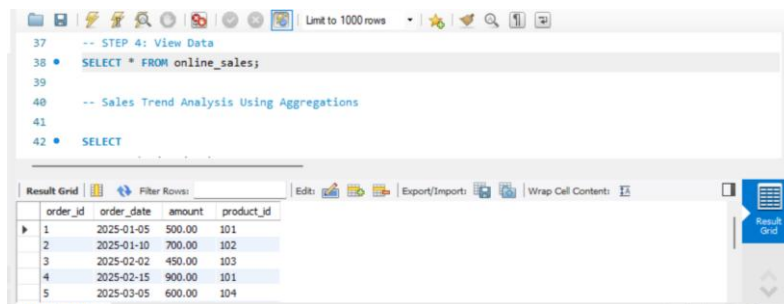
```
CREATE TABLE online_sales (  
order_id INT PRIMARY KEY,  
order_date DATE,  
amount DECIMAL(10,2),  
product_id INT  
);
```

Step 4: Insert Sample Data

```
INSERT INTO online_sales (order_id, order_date, amount, product_id)  
VALUES  
(1,'2025-01-05',500.00,101),  
(2,'2025-01-10',700.00,102),  
(3,'2025-02-02',450.00,103),  
(4,'2025-02-15',900.00,101),
```

```
(5,'2025-03-05',600.00,104),  
(6,'2025-03-18',750.00,102),  
(7,'2025-03-25',1200.00,105),  
(8,'2025-04-08',800.00,101),  
(9,'2025-04-20',950.00,103),  
(10,'2025-05-01',1100.00,104);
```

Output:



The screenshot shows a SQL IDE interface. The top pane contains a SQL query: `-- STEP 4: View Data`, `38 • SELECT * FROM online_sales;`, `39`, `40 -- Sales Trend Analysis Using Aggregations`, `41`, and `42 • SELECT`. The bottom pane displays the 'Result Grid' with 5 rows of data. The columns are 'order_id', 'order_date', 'amount', and 'product_id'. The data rows are: (1, 2025-01-05, 500.00, 101), (2, 2025-01-10, 700.00, 102), (3, 2025-02-02, 450.00, 103), (4, 2025-02-15, 900.00, 101), and (5, 2025-03-05, 600.00, 104).

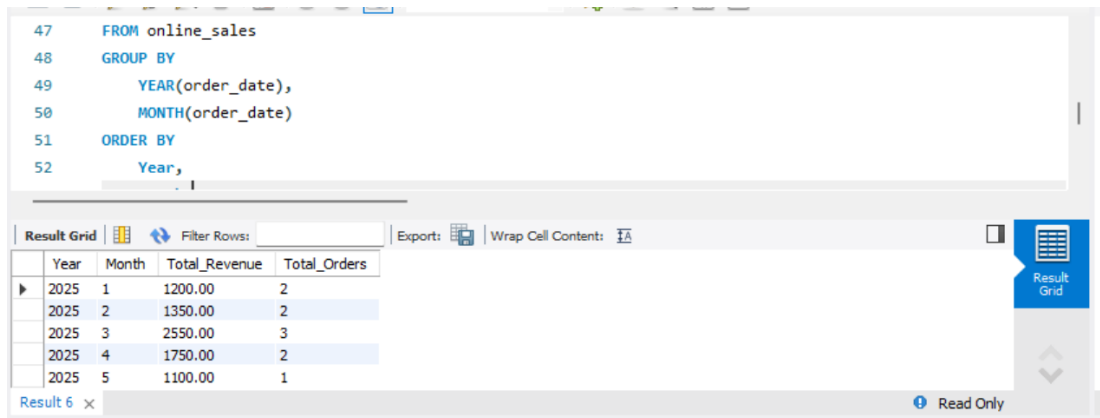
order_id	order_date	amount	product_id
1	2025-01-05	500.00	101
2	2025-01-10	700.00	102
3	2025-02-02	450.00	103
4	2025-02-15	900.00	101
5	2025-03-05	600.00	104

Step 5: Sales Trend Analysis Using Aggregations

```
SELECT  
YEAR(order_date) AS Year,  
MONTH(order_date) AS Month,  
SUM(amount) AS Total_Revenue,  
COUNT(DISTINCT order_id) AS Total_Orders  
FROM online_sales  
GROUP BY  
YEAR(order_date),  
MONTH(order_date)  
ORDER BY  
Year,
```

Month;

Output:



The screenshot shows a SQL query editor with the following query:

```
47 FROM online_sales
48 GROUP BY
49     YEAR(order_date),
50     MONTH(order_date)
51 ORDER BY
52     Year,
```

Below the query editor is a 'Result Grid' showing the output of the query. The grid has columns for Year, Month, Total_Revenue, and Total_Orders. The data is as follows:

	Year	Month	Total_Revenue	Total_Orders
▶	2025	1	1200.00	2
	2025	2	1350.00	2
	2025	3	2550.00	3
	2025	4	1750.00	2
	2025	5	1100.00	1

The interface also includes a 'Filter Rows' section, an 'Export' button, and a 'Wrap Cell Content' option. A 'Result Grid' button is visible on the right side of the interface.

Step 6: Sales Analysis for a Specific Time Period

SELECT

YEAR(order_date) AS Year,

MONTH(order_date) AS Month,

SUM(amount) AS Total_Revenue,

COUNT(DISTINCT order_id) AS Total_Orders

FROM online_sales

WHERE order_date BETWEEN '2025-01-01' AND '2025-03-31'

GROUP BY

YEAR(order_date),

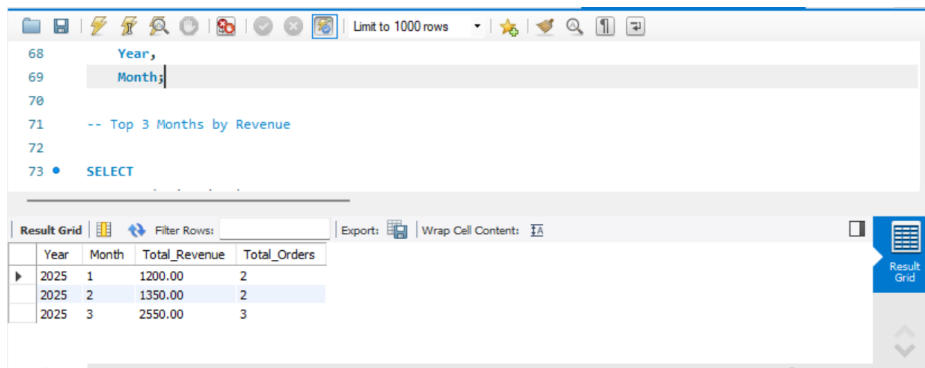
MONTH(order_date)

ORDER BY

Year,

Month;

Output:



68 Year,
69 Month;
70
71 -- Top 3 Months by Revenue
72
73 • SELECT

Result Grid | Filter Rows: | Exports: | Wrap Cell Contents: |

	Year	Month	Total_Revenue	Total_Orders
▶	2025	1	1200.00	2
	2025	2	1350.00	2
	2025	3	2550.00	3

Step 7: Top 3 Months by Revenue

SELECT

YEAR(order_date) AS Year,

MONTH(order_date) AS Month,

SUM(amount) AS Total_Revenue

FROM online_sales

GROUP BY

YEAR(order_date),

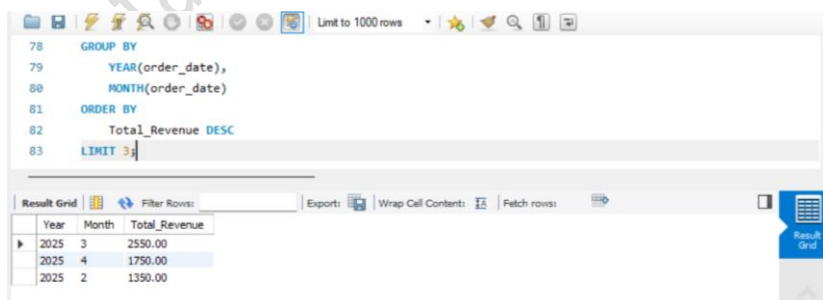
MONTH(order_date)

ORDER BY

Total_Revenue DESC

LIMIT 3;

Output:



78 GROUP BY
79 YEAR(order_date),
80 MONTH(order_date)
81 ORDER BY
82 Total_Revenue DESC
83 LIMIT 3;

Result Grid | Filter Rows: | Exports: | Wrap Cell Contents: | Fetch rows: |

	Year	Month	Total_Revenue
▶	2025	3	2550.00
	2025	4	1750.00
	2025	2	1350.00